

SplashKit Documentation

Team Progress Demo

12 May 2026

Presenter: Ralph Weng

Branch: [demo/leadership-2026-05-12](#)

Team Progress Demo — 2026-05-12

What we shipped this trimester

Eleven student contributors shipped 22 peer-approved PRs since 1 March 2026 — closing out SplashKit's keyboard and mouse input documentation, adding the **first interface widget** (`button`), and introducing a new artefact type: a **generative-AI tutorial guide** for SplashKit learners.

Today walks through **9 of those PRs**, live, on a local build that bundles them into a single staging branch (`demo/leadership-2026-05-12`).

Every contributor wrote, tested, and documented their function in **four languages** — C++, C# (top-level), C# (OOP), and Python — the unit's core competency.

Team & velocity

Metric	This trimester
Open PRs since 1 March	63
Ready to merge (2+ approvals, no change requests)	22
In second review (1 approval)	12
Awaiting first review	6
Unique student contributors to the ready-to-merge tier	11

The 9 PRs in today's demo were selected for **visual variety + breadth across three API areas** (graphics, input, interface) plus one non-code artefact (the AI guide). The remaining 13 ready-to-merge PRs follow the same authoring pattern.

Feature walkthrough

1. `button` — first interface widget

URL: <http://localhost:4321/api/interface/#button>

Button {</>}

See Code Examples

Example 1: Button Click Counter

C++

C#

Python

```

#include "splashkit.h"

int main()
{
    open_window("Button Click Counter", 800, 600);

    // Track click counts for each button
    int red_count = 0;
    int blue_count = 0;
    int green_count = 0;

    while (!quit_requested())
    {
        process_events();
        clear_screen(COLOR_WHITE);

        // Show instructions for the user
        draw_text("Click the buttons to increment counters", COLOR_BLACK, 10, 10);

        // Create a panel with three buttons
        if (start_panel("Click Counter", rectangle_from(50, 50, 200, 200)))
        {
            // Check if each button is clicked and update counts
            if (button("Red"))
            {
                red_count++;
            }
            if (button("Blue"))
            {
                blue_count++;
            }
            if (button("Green"))
            {
                green_count++;
            }
        }
        end_panel("Click Counter");
    }

    // Display each button's click count on the window
    draw_text("Red clicks: " + std::to_string(red_count), COLOR_RED, 300, 100);
    draw_text("Blue clicks: " + std::to_string(blue_count), COLOR_BLUE, 300, 130);
    draw_text("Green clicks: " + std::to_string(green_count), COLOR_GREEN, 300, 160);

    refresh_screen(60);
}

close_all_windows();
return 0;

```

Output:

A first-class interface widget: renders a clickable button with hover and active states, returns `true` on click. Shifts SplashKit's UI offering from "draw your own controls" to "compose with provided primitives." Unblocks every future widget — sliders, checkboxes, dropdowns — that would otherwise need to ship `button` first.

Author: Ralph Weng (@ralphweng-autograb) · PR [#709](#)

2. draw_circle — graphics

URL: <http://localhost:4321/api/graphics/#draw-circle>

Draw Circle (←→)

See Code Examples

Example 1: Circle Showcase

C++C#Python

```
#include "splashkit.h"

int main()
{
    open_window("Draw Circle Example", 800, 600);
    clear_screen(COLOR_WHITE);

    // Draw a large red filled circle in the center
    fill_circle(COLOR_RED, 400, 300, 100);

    // Draw circles with different colors, sizes, and positions
    draw_circle(COLOR_BLUE, 200, 150, 80);
    draw_circle(COLOR_GREEN, 600, 150, 60);
    draw_circle(COLOR_ORANGE, 200, 450, 70);
    draw_circle(COLOR_PURPLE, 600, 450, 65);

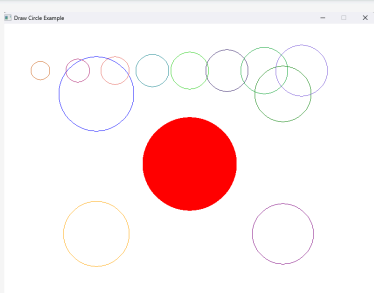
    // Draw smaller circles with varying colors
    for (int i = 0; i < 8; i++)
    {
        int radius = 20 + i * 5;
        int x = 400 + (i - 4) * 80;
        int y = 300;
        draw_circle(random_rgb_color(255), x, y, radius);
    }

    refresh_screen();

    delay(5000);
    close_all_window();

    return 0;
}
```

Output:



Example 2: Animate a circle moving across the screen

C++Python

```
#include "splashkit.h"

int main()
{
    open_window("Circle Animation", 800, 600);

    double x = 0;

    while (!quit_requested())
    {
        process_events();
        clear_screen(COLOR_WHITE);

        draw_circle(COLOR_RED, x, 300, 50);

        x += 2;
        if (x > 800) x = 0;

        refresh_screen(60);
    }

    return 0;
}
```

Output:

Example 3: Move a circle using arrow keys to demonstrate interactive usage of draw_circle.

C++Python

```
#include "splashkit.h"

int main()
{
    open_window("Interactive Circle", 800, 600);

    double x = 400;
    double y = 300;

    while (!quit_requested())
    {
        process_events();
        clear_screen(COLOR_WHITE);

        if (key_down(LEFT_KEY)) x -= 5;
        if (key_down(RIGHT_KEY)) x += 5;
        if (key_down(UP_KEY)) y -= 5;
        if (key_down(DOWN_KEY)) y += 5;

        draw_circle(COLOR_BLUE, x, y, 50);

        refresh_screen(60);
    }

    return 0;
}
```

Output:

Two new variants for `draw_circle` : an animation example (a perpetually growing/shrinking circle) and an interactive example (radius controlled by the mouse). Demonstrates the function on a real visual, not just signature documentation — exactly the kind of "see it, then read about it" a learner needs.

Author: Janki Luitel (@jankiluitel) · PR [#712](#)

3. AI Prompt Helper — generative-AI guide

URL: <http://localhost:4321/guides/generative-ai/ai-prompt-helper-for-splashkit/>

AI Prompt Helper for SplashKit Game Ideas

Learn how to use AI tools responsibly to generate game ideas, debug code, and understand SplashKit concepts.
 Written by: Janki Luitel
 Last updated: 2026-05-03T00:00:00.000Z

Artificial Intelligence (AI) tools can support your learning journey when building games with SplashKit. AI should be used as a learning assistant — not as a replacement for your own problem-solving, coding, and creativity.

Why Use AI with SplashKit?

AI tools can help you:

- Brainstorm game ideas
- Break complex programming tasks into smaller steps
- Understand SplashKit functions
- Debug code errors
- Generate pseudocode before implementation
- Learn programming concepts faster

Example Prompt 1: Game Idea Generation

"Give me 5 beginner game ideas using SplashKit in C++"

Example outputs may include:

- Maze Escape Game
- Space Shooter
- Flappy Bird Clone
- Bouncing Ball Physics Game
- Platform Adventure

Example Prompt 2: Understanding SplashKit Functions

"Explain how draw_circle works in SplashKit with a simple example."

AI can help explain:

- Parameters
- Colours
- Coordinates
- Radius
- Real-world use cases

Example Prompt 3: Debugging

"Why am I getting includePath errors in SplashKit C++?"

AI may help identify:

- Missing SDK paths
- Incorrect compiler settings
- VS Code configuration issues

Example Prompt 4: Pseudocode Planning

"Help me design pseudocode for a SplashKit bouncing ball game."

This can help you plan:

- Variables
- Movement logic
- Collision detection
- Game loop structure

Bad Prompt vs Good Prompt

Bad prompt:

"Fix my SplashKit code"

Good prompt:

"I am using SplashKit in C++. My goal is to draw a circle in a window, but the window closes immediately. Explain why this happens and suggest how to fix it without rewriting the whole program."

Responsible Use of AI

When using AI:

- Use AI to learn concepts
- Ask AI for explanations
- Use AI for brainstorming
- Verify AI-generated code

Avoid:

- Copying code without understanding it
- Submitting AI-generated work as your own
- Relying on AI without testing your code

Final Checklist

Before using AI-generated ideas or code:

- Do I understand how it works?
- Can I explain it to someone else?
- Have I tested it myself?
- Does it follow SplashKit style guidelines?

If yes, you are using AI responsibly.

Happy learning with SplashKit and AI!

Previous
 Routing With Servers

Next
 Overview

A different artefact type — not a usage example, a **tutorial guide page**. Walks SplashKit learners through using AI tools responsibly: prompting patterns for game-idea generation, function lookup, debugging, and pseudocode planning, plus a "good prompt vs bad prompt" comparison and a responsible-use checklist. Establishes a pattern other contributors can replicate.

Author: Janki Luitel (@jankiluitel) · PR [#749](#)

4. Input handling — completing the input surface

Six new examples across five PRs, finishing off SplashKit's keyboard and mouse event coverage. Each example follows the same four-language pattern with a recorded GIF demonstrating the behaviour.

Function	What it adds	Author	PR
<code>key_down</code>	Detect a held-down keyboard key (vs <code>key_typed</code> , which fires once per press)	Jasveena (@jasveena15)	#702
<code>mouse_position</code>	Returns the cursor's (<code>x</code> , <code>y</code>) as a <code>Point2D</code> ; example animates a tracker dot	Osaid (@Osaid2993)	#710
<code>key_typed</code>	Single-press event — powers menu navigation, pause toggles, one-shot weapon fire	Osaid (@Osaid2993)	#732
<code>key_released</code>	Fires when a held key is let go — useful for charge-up / hold-to-aim mechanics	Abdullah (@abdullah12121212)	#733
<code>mouse_shown</code>	Whether the OS cursor is visible — pairs with <code>hide_mouse</code> / <code>show_mouse</code> for cursor-locked games (bundled in #733)	Abdullah (@abdullah12121212)	#733
<code>any_key_pressed</code>	"Press any key to continue" — without it, contributors had to loop every <code>KeyCode</code> . One line vs ~200.	Osaid (@Osaid2993)	#734
<code>key_name</code>	Returns the human-readable name for a <code>KeyCode</code> (<code>"Space"</code> , <code>"Left Arrow"</code>) — for in-game key-binding UIs	Kris Denman (@KayDee6751)	#745

Hero example for the demo: `key_down` . The recorded GIF on the page shows a live keyboard-state display (Left / Right / Up / Down / Space) updating in real time as keys are held.

URL: <http://localhost:4321/api/input/#key-down>

Key Down {</>}

See Code Examples

Example 1: Keyboard State Display

C++

C#

Python

```

#include "splashkit.h"

void draw_key_status(string label, key_code key, double x, double y)
{
    bool pressed = key_down(key);

    color indicator;
    string state;

    if (pressed)
    {
        indicator = COLOR_GREEN;
        state = "Pressed";
    }
    else
    {
        indicator = COLOR_GRAY;
        state = "Released";
    }

    fill_circle(indicator, x, y, 25);
    draw_text(label + ": " + state, COLOR_BLACK, x + 40, y - 10);
}

int main()
{
    open_window("Keyboard State Display", 800, 400);

    while (!quit_requested())
    {
        process_events();

        clear_screen(COLOR_WHITE);

        draw_text("Press the arrow keys or space bar to see their current state.", COLOR_BLACK, 120, 40);

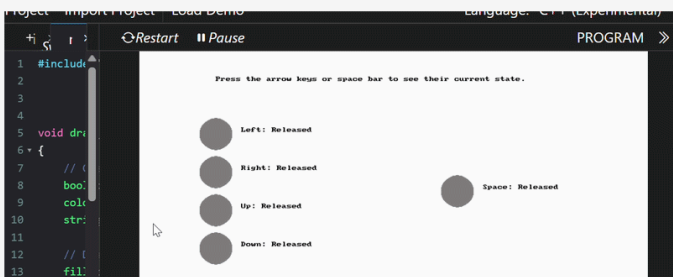
        draw_key_status("Left", LEFT_KEY, 120, 130);
        draw_key_status("Right", RIGHT_KEY, 120, 190);
        draw_key_status("Up", UP_KEY, 120, 250);
        draw_key_status("Down", DOWN_KEY, 120, 310);
        draw_key_status("Space", SPACE_KEY, 500, 220);

        refresh_screen(60);
    }

    return 0;
}

```

Output:



Anticipated questions

"Why aren't these merged to upstream `main` yet?" They've passed peer review (2+ approvals each) and are queued at the `thoth-tech/splashkit.io-starlight` upstream. Peer review and authoring move faster than maintainer merge cadence — that's the bottleneck the program is currently working through.

"What's the maintainer review SLA?" There isn't a formal one. Upstream maintainers merge in batches when they're available. The team's job is to keep the queue full and reviewable; the merge gate is downstream of us.

"How do you pick what gets demoed?" Visual variety + breadth across API areas. The 9 chosen here cover graphics, input, interface, and the new generative-AI guide stream. The other 13 ready-to-merge PRs are more keyboard / geometry / utilities and follow the same authoring pattern.

"How are usage examples authored?" Each example is six files under `public/usage-examples/<category>/` : `.cpp` , `.cs` (top-level), `.cs` (OOP), `.py` , a one-line `.txt` title, and a `.gif` / `.png` / `.webm` output asset. The build pipeline (`scripts/api-pages-script.cjs`) scans these at `npm run build` time and auto-injects them into the API reference page as a tabbed code block plus the visual.

"Test coverage?" Code samples are compile-checked at build time via `usage-examples-testing-script.cjs` . Visual output (the GIFs) is verified by human reviewers — that's a known gap, currently caught at the peer-review stage.

"How many PRs in flight overall?" 63 open since trimester start. 22 ready to merge, 12 awaiting a second approval, 6 awaiting first review. The remainder are blocked on author follow-up after a change request.

What's next

- **Clear the remaining 13 ready-to-merge PRs** to upstream by end of trimester so the public docs reflect current team output.
- **Onboard 2 new contributors to the AI-guide pattern** Janki established — extends our coverage beyond pure code samples.
- **Begin geometry and audio coverage** — currently 0 student-authored usage examples in those categories despite both being core SplashKit surface area.

Note on local search: the search box at the top of the local preview will not return results — the Algolia index is only built against the production site at splashkit.io. Switch to live if a search demo is requested.

This document corresponds to local branch `demo/leadership-2026-05-12` built at 2026-05-12. Backup at github.com/ralphweng2023/splashkit.io-starlight.